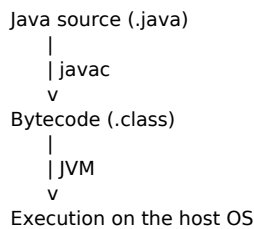


- JVM: Executes bytecode and provides runtime services such as memory management and class loading.
- JRE: Historical term for JVM plus runtime libraries. Modern Java distributions generally focus on the JDK rather than a separate JRE download.
- JDK: Development kit containing compiler and development tools plus the runtime.



Core language topics

- Primitive vs reference types
- Variables and operators
- Conditions and loops
- Arrays
- Methods and constructors
- static, final, this, super
- Packages and access modifiers

Strings

```

String s = "Java";
StringBuilder sb = new StringBuilder("Java");
sb.append(" Backend");
System.out.println(sb);
  
```

Interview Q&A;

Q: Why is Java platform independent?

A: Java source is compiled to platform-neutral bytecode. A JVM implementation for each operating system executes that bytecode.

Q: What is bytecode?

A: The intermediate .class representation produced by the Java compiler. The JVM loads and executes it.

Q: Why is String immutable?

A: Once created, a String object's contents cannot change. This supports safe sharing, predictable hashing, string pooling and thread-safety of the value itself.

Q: String vs StringBuilder vs StringBuffer?

A: String is immutable. StringBuilder is mutable and generally preferred for single-threaded string construction. StringBuffer is mutable and synchronized, so it may have more overhead.

Q: == vs equals()?

A: For primitives, == compares values. For object references, == compares identity. equals() is for logical equality when the class implements it appropriately.

Q: What does final mean?

A: A final variable cannot be reassigned; a final method cannot be overridden; a final class cannot be extended.

Q: What is static?

A: A static member belongs to the class rather than an individual object instance.

2. OOP Concepts with Examples

The four pillars are encapsulation, inheritance, polymorphism and abstraction.

Encapsulation

```

class Student {
    private int age;
  
```

```

    public void setAge(int age) {
        if (age >= 0) this.age = age;
    }

    public int getAge() {
        return age;
    }
}

```

Inheritance

```

class Animal {
    void eat() { System.out.println("Eating"); }
}

class Dog extends Animal {
    void bark() { System.out.println("Barking"); }
}

```

Polymorphism

```

class Printer {
    void print(int x) {}
    void print(String s) {}
}

Animal a = new Dog();
a.eat();

```

Abstraction

```

interface Payment {
    void pay(double amount);
}

class CardPayment implements Payment {
    public void pay(double amount) {
        System.out.println(amount);
    }
}

```

Interview Q&A;

Q: What are the four pillars of OOP?

A: Encapsulation, inheritance, polymorphism and abstraction.

Q: Overloading vs overriding?

A: Overloading uses the same method name with different parameter lists and is resolved at compile time. Overriding changes inherited behavior with a compatible method signature and is selected at runtime.

Q: Abstract class vs interface?

A: An abstract class can hold state, constructors and implemented methods. An interface defines a contract and supports multiple implementation inheritance; modern interfaces can also contain default and static methods.

Q: What is runtime polymorphism?

A: A parent reference can refer to a child object, and an overridden method is selected based on the runtime object's actual class.

Q: What is composition?

A: A class contains another object and delegates work to it. It represents a HAS-A relationship and often reduces tight coupling compared with deep inheritance.

Q: IS-A vs HAS-A?

A: IS-A is inheritance, such as Dog IS-A Animal. HAS-A is composition/association, such as Car HAS-A Engine.

3. Collections & Generics

```

List -> ArrayList, LinkedList
Set -> HashSet, LinkedHashSet, TreeSet
Queue -> Queue / Deque implementations
Map -> HashMap, LinkedHashMap, TreeMap

```

Interview Q&A;

Q: ArrayList vs LinkedList?

A: ArrayList is a dynamic array with fast indexed access and is the common default List choice. LinkedList uses linked nodes and can be useful for specific deque/linked-list operations, but it is not automatically faster for arbitrary insertion/removal.

Q: HashMap vs HashSet?

A: HashMap stores key-value pairs. HashSet stores unique values and uses hashing for membership checks.

Q: HashMap vs TreeMap?

A: HashMap provides hash-based lookup without sorted-key ordering guarantees. TreeMap keeps keys sorted and supports navigable operations, typically at $O(\log n)$ for core operations.

Q: What happens when two HashMap keys collide?

A: They map to the same bucket. The map distinguishes entries using hash comparisons and equals() on keys; modern implementations can use tree structures for sufficiently collision-heavy buckets.

Q: Why must equals() and hashCode() agree?

A: If two objects are logically equal, they must return the same hash code; otherwise hash-based collections may fail to locate equal keys/elements correctly.

Q: Comparable vs Comparator?

A: Comparable defines a type's natural ordering via compareTo(). Comparator defines an external/custom ordering and is useful when multiple orderings are needed.

Q: Why use generics?

A: Generics provide compile-time type safety and reduce explicit casting, e.g. List prevents adding an Integer.

4. Java 8 Features with Practicals

High-value Java 8 features: lambda expressions, functional interfaces, method references, Stream API, Optional, default/static interface methods, and the modern Date/Time API.

Lambda

```
List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5, 6);  
  
nums.forEach(n -> System.out.println(n));
```

Stream practical

```
List<Integer> evens = nums.stream()  
    .filter(n -> n % 2 == 0)  
    .map(n -> n * 10)  
    .distinct()  
    .sorted()  
    .collect(Collectors.toList());
```

Interview Q&A;

Q: What is a lambda expression?

A: A concise way to provide behavior for a functional interface's single abstract method.

Q: What is a functional interface?

A: An interface with exactly one abstract method. It may still have default and static methods. @FunctionalInterface lets the compiler verify the intended contract.

Q: What is Stream API?

A: A pipeline abstraction for processing data using operations such as filter, map, sorted, reduce and collect. It does not itself store the data.

Q: map vs filter?

A: map transforms each element into another value; filter keeps or removes elements based on a predicate.

Q: map vs flatMap?

A: map produces one result per input. flatMap is used when each input produces a stream/collection and you want to flatten the nested results into one stream.

Q: Why are streams lazy?

A: Intermediate operations generally build a pipeline without processing elements immediately. A terminal operation triggers traversal, allowing optimizations such as avoiding unnecessary work.

Q: What is Optional?

A: A container type intended to model the presence or absence of a value and make missing-value handling explicit. It is most useful for return values and should not be treated as a universal replacement for null.

Q: What is a method reference?

A: A compact syntax for passing an existing method as behavior, e.g. `System.out::println`.

5. Java 9-21: Important Features

For interview purposes, focus on the major language milestones rather than memorizing every JDK API change. The following summary follows Oracle's Java language-change documentation.

`□cite□turn0search0□turn0search1□`

Version	Feature	Interview takeaway
9	Modules	Strongly encapsulated module boundaries; requires/exports.
10	var	Local-variable type inference; still statically typed.
11	Lambda var parameters	Useful when annotations/modifiers are needed in lambda parameters.
12-13	Switch preview	Evolution toward switch expressions.
14	Switch expressions	switch can produce a value; arrow labels reduce fall-through.
15	Text blocks	Readable multiline string literals.
16	Records; instanceof patterns	Compact data carriers and less casting boilerplate.
17	Sealed classes	Restrict permitted subclasses/implementations.
18	UTF-8 default	UTF-8 became the default charset for standard Java APIs where specified.
19-20	Preview evolution	Virtual threads and pattern features advanced through previews.
21	Record patterns; switch patterns; virtual threads	Major modern Java release and LTS.

Examples

```
var name = "Java";

record Student(int id, String name) {}

sealed interface Shape permits Circle, Rectangle {}
final class Circle implements Shape {}
final class Rectangle implements Shape {}

String result = switch (obj) {
  case String s -> "String: " + s;
  case Integer i -> "Integer: " + i;
  case null -> "null";
  default -> "Other";
};
```

Interview Q&A;

Q: Is var dynamically typed?

A: No. var uses compile-time local-variable type inference. Once inferred, the variable has a static type.

Q: What problem do records solve?

A: They provide a concise syntax for data-carrier classes and automatically supply accessors, equals, hashCode and toString based on record components.

Q: What are sealed classes?

A: They restrict which classes or interfaces may directly extend or implement a type, making a closed hierarchy possible.

Q: What are switch expressions?

A: A switch can produce a value and can use arrow labels, reducing accidental fall-through.

Q: What are text blocks?

A: Multiline string literals that make embedded JSON, SQL, HTML and other structured text easier to read.

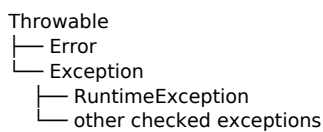
Q: What are virtual threads?

A: Lightweight threads designed to support very high concurrency, especially for workloads involving blocking I/O. They became a permanent feature in Java 21.

Q: Which modern Java features should an intern know first?

A: Records, var, switch expressions, pattern matching, sealed classes and the basic idea of virtual threads. Prioritize Java 8 streams/lambda and core Java even more heavily.

6. Exception Handling



Interview Q&A;

Q: Checked vs unchecked exception?

A: Checked exceptions are subject to compiler enforcement for catch-or-declare rules. Unchecked exceptions are RuntimeException subclasses and do not have that compiler requirement.

Q: throw vs throws?

A: throw explicitly throws an exception instance. throws declares that a method may propagate specified exception types.

Q: finally?

A: A block that normally executes after try/catch for cleanup, subject to abrupt termination such as JVM shutdown or certain fatal failures. Prefer try-with-resources for resource management.

Q: What is try-with-resources?

A: A construct that automatically closes AutoCloseable resources, reducing resource-leak risk.

Q: Should you catch Exception everywhere?

A: No. Catch the most specific exception you can meaningfully handle. Broad catches can hide programming errors and make diagnosis harder.

Q: How do you create a custom exception?

A: Extend an appropriate exception class, usually RuntimeException for application/business exceptions where callers do not need mandatory catch/declare handling.

Practical

```

class UserNotFoundException extends RuntimeException {
    public UserNotFoundException(String message) {
        super(message);
    }
}
  
```

7. Concurrency & Deadlock

Thread A: Lock 1 → waits for Lock 2
Thread B: Lock 2 → waits for Lock 1

Four classic deadlock conditions

- Mutual exclusion
- Hold and wait
- No preemption
- Circular wait

Interview Q&A;

Q: What is a deadlock?

A: A state where threads wait indefinitely because each needs a resource/lock held by another thread.

Q: How can you prevent deadlock?

A: Use consistent lock ordering, minimize lock scope, avoid unnecessary nested locks, reduce shared mutable state, and use timed locking where appropriate.

Q: Race condition vs deadlock?

A: A race condition is incorrect or nondeterministic behavior caused by unsafe concurrent access. A deadlock is indefinite waiting between threads.

Q: What does synchronized provide?

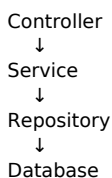
A: Mutual exclusion for the protected monitor and memory-visibility/happens-before guarantees around monitor operations.

Q: What does volatile do?

A: It provides visibility and ordering guarantees for reads/writes of the variable, but does not make compound operations such as `count++` atomic.

8. Spring & Spring Boot Basics

Spring's core idea is IoC: the framework manages objects (beans) and their dependencies. Spring Boot adds conventions, auto-configuration and starters to make Spring applications easier to build and run.



Common annotations

- `@Component` — generic bean
- `@Service` — business/service layer
- `@Repository` — persistence layer
- `@RestController` — REST controller
- `@Configuration` — configuration class
- `@Bean` — explicit bean definition

Interview Q&A;

Q: What is dependency injection?

A: A class receives its dependencies from outside rather than constructing them itself. This reduces coupling and improves testability.

Q: Why constructor injection?

A: Dependencies are explicit, can be final, and are available when the object is created. It also makes unit testing straightforward.

Q: What is a Spring bean?

A: An object managed by the Spring container/ApplicationContext.

Q: @Component vs @Service vs @Repository?

A: All are component stereotypes used for component scanning; Service communicates business intent and Repository communicates persistence intent, while Component is generic.

Q: What does @SpringBootApplication represent?

A: It combines configuration, component scanning and Spring Boot auto-configuration behavior.

Q: What is auto-configuration?

A: Spring Boot attempts to configure application components based on the classpath, existing beans and configuration properties, reducing boilerplate setup.

9. REST APIs

HTTP	Typical purpose
GET	Read
POST	Create / process
PUT	Replace/update
PATCH	Partial update
DELETE	Delete

```
@RestController
@RequestMapping("/users")
class UserController {

    @GetMapping("/{id}")
    public User get(@PathVariable Long id) {
        return service.findById(id);
    }

    @PostMapping
    public ResponseEntity<User> create(@RequestBody User user) {
        return ResponseEntity.status(201)
            .body(service.create(user));
    }
}
```

Interview Q&A;

Q: What is REST?

A: An architectural style for resource-oriented network communication. HTTP methods, resource URLs and representations such as JSON are commonly used in REST APIs.

Q: Path variable vs query parameter?

A: A path variable identifies a resource, e.g. /users/10. A query parameter modifies/filter the request, e.g. /users?role=admin.

Q: 401 vs 403?

A: 401 indicates the request lacks valid authentication credentials. 403 means access is forbidden even though the request is understood/authenticated as applicable.

Q: What is idempotency?

A: An operation is idempotent when repeating the same request has the same intended effect as performing it once. GET, PUT and DELETE are designed with idempotent semantics; POST is generally not.

Q: Why use DTOs?

A: DTOs define an API contract independent of persistence entities, control exposed fields and support validation/versioning more cleanly.

10. Global Exception Handling

```
@RestControllerAdvice
class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    ResponseEntity<?> notFound(UserNotFoundException ex) {
        return ResponseEntity.status(404)
            .body(Map.of("message", ex.getMessage()));
    }
}
```

Interview Q&A;

Q: Why global exception handling?

A: It centralizes error-to-HTTP-response mapping, avoids repeated try/catch logic in controllers, and creates consistent API error responses.

Q: @ExceptionHandler vs @RestControllerAdvice?

A: @ExceptionHandler identifies a handler for a specific exception. @RestControllerAdvice applies controller advice globally (or to selected controllers) and commonly combines with exception handlers to produce REST responses.

Q: What should an API error response contain?

A: Typically status, a safe human-readable message, timestamp/correlation information where useful, and optionally validation details. Avoid exposing stack traces, secrets or database internals.

11. JPA & Hibernate

JPA (Jakarta Persistence) is a specification. Hibernate ORM is a widely used implementation that provides ORM functionality and implements JPA APIs.

```
@Entity
class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
}
```

Interview Q&A;

Q: JPA vs Hibernate?

A: JPA defines standard persistence APIs and mappings; Hibernate is an implementation of those APIs plus additional ORM capabilities.

Q: What is an entity?

A: A persistent domain object mapped to database data, normally annotated with @Entity and having an identifier.

Q: What is a persistence context?

A: A set of managed entity instances associated with an EntityManager. It provides identity tracking and enables dirty checking.

Q: What is dirty checking?

A: Hibernate can detect changes to managed entities and synchronize those changes to the database during flush/transaction processing.

Q: Lazy vs eager loading?

A: Lazy loading defers fetching a relationship until needed. Eager loading fetches it earlier. Lazy is often preferable for large relationships, but access must be planned to avoid lazy-loading errors or excessive queries.

Q: What is the N+1 problem?

A: A query loads N parent records, then additional queries are executed for each parent to load related data. It can cause severe performance problems.

Q: How can you reduce N+1?

A: Use appropriate fetch joins/entity graphs or carefully designed queries, batch fetching where appropriate, and measure generated SQL rather than blindly switching everything to eager.

12. Hibernate Relationships

Relation	Example	Annotation
One-to-One	Person ↔ Passport	@OneToOne
One-to-Many	Department → Employees	@OneToMany
Many-to-One	Employees → Department	@ManyToOne
Many-to-Many	Students ↔ Courses	@ManyToMany

```

@Entity
class Employee {
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")
    private Department department;
}

@Entity
class Department {
    @OneToMany(mappedBy = "department")
    private List<Employee> employees;
}

```

Interview Q&A;

Q: What does mappedBy mean?

A: It identifies the field on the owning side that manages the relationship. It prevents the inverse side from creating an additional mapping.

Q: Which side is usually the owning side in Department-Employee?

A: The Employee's @ManyToOne side commonly owns the foreign-key mapping because the employee table contains department_id.

Q: What is @JoinColumn?

A: It specifies the database column used to join the relationship, commonly a foreign-key column.

Q: What is cascade?

A: It controls which persistence operations propagate from one entity to associated entities. Use it deliberately; CascadeType.ALL is not automatically correct.

Q: Why avoid direct many-to-many in some real systems?

A: If the relationship itself has attributes such as enrollment date or status, a separate join entity is usually clearer and more flexible.

13. Spring Security & JWT

Authentication answers 'who are you?'; authorization answers 'what are you allowed to do?'. Spring Security provides the authentication/authorization framework. Spring Boot's reference documentation describes default web security behavior and customization. [\[cite\]turn0search9](#)

```

Login
↓
Authenticate credentials
↓
Issue access token (commonly JWT)
↓
Client sends Authorization: Bearer <token>
↓
Security filters validate token
↓
Authorized endpoint

```

Interview Q&A;

Q: What is JWT?

A: A compact token format carrying claims, commonly signed to provide integrity/authenticity. A JWT payload is not automatically encrypted, so secrets should not be placed in it.

Q: What should be validated in a JWT?

A: Signature, expiration and relevant claims such as issuer/audience, depending on the application.

Q: Authentication vs authorization?

A: Authentication verifies identity; authorization evaluates permissions/roles for a requested action.

Q: Where should passwords be stored?

A: Never in plaintext. Store strong password hashes using a suitable password hashing algorithm such as BCrypt or Argon2 via an appropriate PasswordEncoder.

Q: What is SecurityFilterChain?

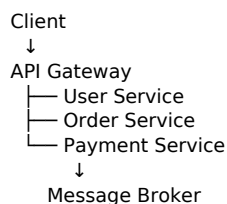
A: It defines the web-security filter behavior and authorization rules for incoming requests in modern Spring Security configurations.

Q: 401 vs 403 in security?

A: 401 is used when authentication is missing/invalid. 403 is used when the request is authenticated but the principal is not permitted to access the resource.

14. Microservices Theory

Microservices architecture organizes a system as independently deployable services around cohesive business capabilities. It improves independent scaling and team autonomy but adds distributed-systems complexity.



Interview Q&A;

Q: Monolith vs microservices?

A: A monolith is one deployable application. Microservices split the system into independently deployable services. Microservices provide scaling and deployment independence but add network, observability and consistency complexity.

Q: Why microservices?

A: Independent deployment/scaling, clearer business boundaries, team autonomy and isolation of certain technology/data concerns.

Q: When should you NOT use microservices?

A: When the system is small, the team is inexperienced with distributed operations, or service boundaries are unclear. A modular monolith can be a better starting point.

Q: What is service discovery?

A: A mechanism allowing services to locate available instances dynamically rather than hard-coding network addresses.

Q: What is an API Gateway?

A: A client-facing entry point that can route requests and centralize concerns such as authentication, rate limiting, aggregation or observability.

Q: What is eventual consistency?

A: A distributed system may temporarily expose different states across services, with replicas/services converging toward a consistent state over time.

15. Microservices Patterns

API Gateway

Routes client traffic and can centralize cross-cutting concerns.

Circuit Breaker

Stops calls to a failing dependency temporarily to prevent cascading failure.

Retry with backoff

Retries transient failures carefully; avoid unsafe repeated non-idempotent writes.

Bulkhead

Isolates resources so one overloaded dependency does not exhaust all resources.

Saga

Coordinates distributed business transactions using local transactions and compensating actions.

Idempotency

Makes repeated delivery/request processing safe against unintended duplicate effects.

Event-driven architecture

Uses asynchronous events to decouple producers and consumers.

Distributed tracing

Tracks an end-to-end request across service boundaries using trace/correlation identifiers.

Interview Q&A;

Q: What is a circuit breaker?

A: A resilience mechanism with states such as closed/open/half-open that can stop calls to a failing dependency and later test recovery.

Q: What is Saga?

A: A distributed transaction pattern in which each service performs a local transaction and publishes/handles steps; failures are handled using compensating actions.

Q: Why is idempotency important?

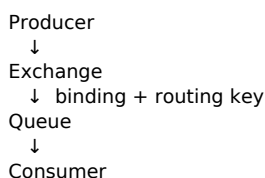
A: Network retries and message redelivery can cause duplicate requests. Idempotency allows repeated processing to have one intended business effect.

Q: What is a bulkhead pattern?

A: It isolates pools/resources so failure or saturation in one dependency does not consume the resources needed by unrelated operations.

16. RabbitMQ

RabbitMQ is a message broker. Its official tutorials cover queues, work queues, publish/subscribe, routing, topics and RPC, including Java examples. [\[cite turn0search7 turn0search8\]](#)



Core components

- Producer — publishes a message.
- Exchange — receives and routes messages.
- Queue — stores messages for consumers.
- Binding — connects an exchange to a queue.
- Routing key — helps routing decisions.
- Consumer — processes messages.
- Acknowledgement — consumer confirms processing.

Interview Q&A;

Q: Why use RabbitMQ?

A: To decouple components, process work asynchronously, buffer bursts and build event-driven integrations.

Q: Exchange vs queue?

A: An exchange routes published messages. A queue stores messages until consumers process them.

Q: What are common exchange types?

A: Direct, fanout, topic and headers exchanges. Direct uses exact routing keys, fanout broadcasts, and topic supports pattern-based routing.

Q: What is an acknowledgement?

A: A consumer acknowledgement tells the broker that a message has been successfully processed. It can help support redelivery when processing fails.

Q: What is a dead-letter queue?

A: A queue used to hold messages that are rejected, expire, exceed configured delivery limits or otherwise become dead-lettered according to broker configuration.

Q: Why should consumers be idempotent?

A: A message may be redelivered. Idempotent processing prevents a duplicate delivery from causing an unintended duplicate business effect.

Spring example

```
@RabbitListener(queues = "orders")
public void consume(OrderCreatedEvent event) {
    // process event
}
```

17. SQL — Basics to Advanced

Master SQL in layers: CRUD → filtering/aggregation → joins → subqueries/CTEs → window functions → transactions/indexes/performance.

CRUD

```
SELECT * FROM employees;
```

```
INSERT INTO employees(name, salary)
VALUES ('Asha', 50000);
```

```
UPDATE employees
SET salary = 55000
WHERE id = 1;
```

```
DELETE FROM employees
WHERE id = 1;
```

Joins and aggregation

```
SELECT d.name, COUNT(e.id) AS employee_count
FROM departments d
LEFT JOIN employees e
    ON e.department_id = d.id
GROUP BY d.id, d.name
ORDER BY employee_count DESC;
```

Window function

```
SELECT name, department_id, salary,
       DENSE_RANK() OVER (
           PARTITION BY department_id
           ORDER BY salary DESC
       ) AS salary_rank
FROM employees;
```

Interview Q&A;

Q: WHERE vs HAVING?

A: WHERE filters rows before grouping. HAVING filters groups after GROUP BY and aggregation.

Q: INNER JOIN vs LEFT JOIN?

A: INNER JOIN returns matching rows from both sides. LEFT JOIN returns every left row and matching right data, with NULL for unmatched right rows.

Q: Primary key vs foreign key?

A: A primary key uniquely identifies a row in its table. A foreign key references a key in another table and enforces a relationship when constraints are configured.

Q: What is normalization?

A: A relational design technique that reduces redundancy and update anomalies by organizing data into related tables according to dependency rules.

Q: What is an index?

A: A data structure that provides an alternate access path to rows. It can speed reads but consumes storage and adds write/update overhead.

Q: What is a transaction?

A: A logical unit of database work whose operations are committed or rolled back together according to transaction semantics.

Q: Explain ACID.

A: Atomicity: all-or-nothing transaction effects. Consistency: valid state transitions under rules/constraints. Isolation: controls interaction among concurrent transactions. Durability: committed changes survive according to database guarantees.

Q: What are window functions?

A: Functions that compute values across related rows while retaining individual rows, using OVER() with optional PARTITION BY and ORDER BY.

Q: ROW_NUMBER vs RANK vs DENSE_RANK?

A: ROW_NUMBER gives unique sequential numbers. RANK gives ties the same rank with gaps. DENSE_RANK gives ties the same rank without gaps.

Q: How do you find the second-highest salary?

A: Use a distinct salary ranking/subquery or a window function such as DENSE_RANK and filter for rank 2; account for duplicate salary values depending on the requirement.

Advanced SQL practice

- Second/Nth highest salary
- Duplicate rows
- Employees without departments
- Department-wise counts
- Top 2 salaries per department
- Customers with more than N orders
- Latest record per customer using ROW_NUMBER
- Running totals using SUM() OVER(...)

18. Cross-Topic Interview Questions

Q: Explain your Java backend request flow.

A: Client sends HTTP request → controller validates/maps input → service applies business rules → repository accesses persistence → response DTO is returned. Exceptions are mapped centrally.

Q: Why should business logic not live in controllers?

A: Controllers should focus on HTTP concerns. Keeping business logic in services improves separation of concerns, reuse and testing.

Q: How would you design a user CRUD API?

A: Define DTOs and endpoints, validate requests, implement controller/service/repository layers, map entities, handle not-found/conflict/validation errors globally, secure endpoints where needed, and test the API.

Q: How would you handle a slow database query?

A: Inspect the query plan and generated SQL, check indexes/selectivity, reduce unnecessary columns/joins, address N+1 queries, review data volume and measure again.

Q: How would you handle a failing downstream service?

A: Use timeouts, bounded retries with backoff where safe, circuit breaker, fallback where meaningful, metrics/logging/tracing, and idempotency for retried operations.

Q: Why use asynchronous messaging?

A: When immediate synchronous coupling is unnecessary, messaging can decouple services, absorb bursts and process work asynchronously.

Q: How does Hibernate map an object to a table?

A: Entity metadata maps class fields to columns and relationships to foreign keys/join tables; Hibernate translates entity operations into SQL and manages persistence state.

Q: What happens when an exception reaches a Spring controller?

A: If not handled locally, Spring's exception resolution mechanisms can route it to an appropriate `@ExceptionHandler`/global advice and produce an HTTP response.

Q: What is the difference between authentication and authorization in an API?

A: Authentication verifies the caller's identity. Authorization decides whether that identity has permission for the requested resource/action.

Q: What should you know before saying 'microservices' in an interview?

A: Be ready to discuss service boundaries, network failure, observability, data ownership/consistency, deployment, resilience patterns and why microservices are justified.

19. Practical Coding & Mini-Tasks

Before the interview, be able to code these without copying a solution:

- Reverse a string and check palindrome.
- Find second largest without sorting.
- Find duplicate elements using a HashSet.
- Count frequencies using HashMap.
- Find missing number using sum or XOR.
- Move zeros to the end in-place.
- Binary search a sorted array.
- Check whether an array is sorted.
- Find intersection of two arrays.
- Write a CRUD Spring Boot REST API.
- Write a global exception handler.
- Write a SQL JOIN + GROUP BY query.
- Find top 2 salaries per department using a window function.

Interview expectation

For each coding solution, be ready to state approach → time complexity → space complexity → edge cases. If you know a brute-force solution, also think about a HashMap/HashSet, sorting, two-pointer or binary-search optimization where appropriate.

20. One Project to Tie Everything Together

Build a small Employee/Task Management REST API. It does not need to be production-scale; the purpose is to demonstrate that you understand the stack end-to-end.

Suggested stack

- Java 17 or Java 21
- Spring Boot
- Spring Web
- Spring Data JPA / Hibernate
- MySQL or PostgreSQL
- Bean Validation
- Spring Security + JWT

- RabbitMQ for one asynchronous event

Architecture

```

Client
↓
REST Controller
↓
Service
↓
Repository
↓
JPA/Hibernate
↓
Database

```

Optional:
Service → RabbitMQ → Consumer

Project interview Q&A;

Q: Why did you choose this architecture?

A: It separates HTTP concerns, business logic and persistence, making the code easier to test and maintain.

Q: Where is validation handled?

A: Request DTO validation can be handled at the API boundary, while business-rule validation belongs in the service/domain layer.

Q: How did you handle errors?

A: Custom exceptions represent expected application failures; global exception handling maps them to consistent HTTP responses.

Q: How did you secure the API?

A: Authenticate users at login, issue a short-lived signed access token, validate it on protected requests, and enforce authorization rules for roles/permissions.

Q: Why RabbitMQ in this project?

A: For a task such as sending notifications or processing an audit/event asynchronously, messaging avoids forcing the API request to wait for the downstream work.

Q: What would you improve next?

A: Add automated tests, structured logging, metrics/tracing, stronger API documentation, rate limiting where needed, database migration tooling, CI/CD and better observability.

21. Last-Day Rapid Revision Sheet

Java: JVM/JDK, String immutability, == vs equals, final/static, collections, HashMap, Comparable/Comparator.

OOP: 4 pillars, overloading/overriding, interface/abstract class, composition.

Java 8: lambda, functional interface, streams, map/filter/reduce, Optional, method reference.

Java 9–21: modules, var, switch expressions, text blocks, records, pattern matching, sealed classes, virtual threads.

Concurrency: race condition, synchronized, volatile, executor, deadlock + prevention.

Spring: IoC, DI, beans, stereotypes, constructor injection, auto-configuration.

REST: HTTP methods, status codes, DTOs, idempotency, path/query parameters.

Exceptions: checked/unchecked, throw/throws, try-with-resources, @RestControllerAdvice, @ExceptionHandler.

Hibernate: JPA vs Hibernate, entity, persistence context, dirty checking, lazy/eager, N+1, relationships.

Security: authentication vs authorization, JWT, password hashing, SecurityFilterChain, 401/403.

Microservices: boundaries, gateway, discovery, circuit breaker, retry, saga, bulkhead, idempotency, tracing.

RabbitMQ: producer, exchange, routing key, queue, consumer, ack, DLQ, redelivery/idempotency.

SQL: joins, group/having, subqueries, CTEs, indexes, transactions/ACID, window functions.

Golden rule: Do not only memorize definitions. For every topic, be able to explain one example, one trade-off and one failure/edge case.